

Network / PIJ 方法归属重构方案

根据反馈收紧后的第二版实施计划

当前代码核查、文件归属判断与 Codex 实施边界

基线: `code(2).zip`, 重点目录为 `mignet_ce/networks` 与 `mignet_ce/pij`

本版以用户最新反馈为最终约束: **不改名、不移动 `common.py` 与 `cost_fusion.py`; 不整理 `legacy`; 不建立额外的方法共享架构。** 仅保留三项实际重构: 把 Network 私有 GRN 逻辑并回所属方法、把 block KL 交还 `compare_N_kl`、把纯数学工具按原文件移动到一个最小化的内部目录。

生成日期: 2026 年 7 月 20 日

项目: 2026 因果涌现

文档类型: Codex 实施计划 / 代码结构验收依据 (反馈修订版)

重构原则: 不改变算法、不改变 registry key、不改变默认参数与输出语义

目录

1 结论先行：本版只做三类修改	2
2 反馈决策矩阵	2
3 Network：把 GRN 私有逻辑并回所属方法	3
3.1 当前问题	3
3.2 目标修改	3
4 PIJ：把 block KL 交还 compare_N_kl	4
4.1 为什么 block KL 是方法私有逻辑	4
4.2 最小化改造方式	4
4.3 必须保持的数值语义	5
5 Compare 数学工具：只移动，不重写	6
5.1 本步骤的准确边界	6
5.2 为什么本版不再合并 distances/cosine/kl	6
5.3 允许修改的内容	7
6 明确保持原样的部分	7
6.1 common.py	7
6.2 cost_fusion.py	7
6.3 legacy	7
6.4 registry 与 package export	8
7 修订后的目标目录	8
7.1 Network	8
7.2 PIJ compare	8
7.3 PIJ legacy	9
8 文件级 Codex 实施步骤	9
8.1 步骤 0：建立重构前快照	9
8.2 步骤 1：合并 Network GRN 私有逻辑	9

8.3	步骤 2: 把 block KL 交还 compare_N_kl	10
8.4	步骤 3: 移动五个数学工具文件	10
8.5	步骤 4: 只清理本轮产生的旧路径	10
8.6	步骤 5: 验证 registry, 而不是重构 registry	11
9	测试计划	11
9.1	导入与编译测试	11
9.2	Network 数值回归	11
9.3	compare_N_kl 回归	11
9.4	数学工具移动回归	12
9.5	legacy 保护性测试	12
10	Codex 不得做的事情	12
11	完成标准	13
12	最终建议	13

1. 结论先行：本版只做三类修改

根据最新反馈，本轮重构不再追求统一所有目录风格，也不再把所有辅助文件重新分层。目标从“全面整理”收紧为“修复真正的归属问题，同时尽量少动代码”。

最终保留的三项修改：

- 1. 将 `mignet_ce/networks/grn_state.py` 并入 `mignet_ce/networks/light_cci_grn.py`;
- 2. 将 `mignet_ce/pij/compare/block_kl.py` 的逻辑交还 `mignet_ce/pij/compare/compare_N_kl.py`;
- 3. 将 `compare` 中纯数学、纯特征和纯优化器工具按原文件移动到最小化的 `compare/_shared/`，不合并公式、不重写实现。

明确取消的修改：

- 不把 `common.py` 改名为 `base.py`;
- 不把 `cost_fusion.py` 改名为 `cost_fusion_base.py`;
- 不移动或重组 `pij/legacy` 下的任何文件;
- 不重做 `registry` 结构，不调整 `registry key`，不扩大 `package export`;
- 不再建立覆盖 PIJ 全体系的统一 `shared/base` 架构。

这里需要特别区分“文件不移动”和“文件绝对不编辑”。`common.py` 的路径、名称和总体职责保持不变，但为了把 block KL 从公共流程中交还给 `compare_N_kl`，允许对 `common.py` 做一处最小化的 hook 改造；除此之外，不对其结构做清理或拆分。

2. 反馈决策矩阵

对象	最终决定	本版执行方式
<code>networks/grn_state.py</code>	合并	并入 <code>light_cci_grn.py</code> ，随后删除原文件。
<code>compare/block_kl.py</code>	合并	并入 <code>compare_N_kl.py</code> ，由该方法覆盖通用 KL cost hook。
<code>compare/common.py</code>	保留	文件名、路径和基类职责不变；仅为 block KL 归属调整加入最小 hook。

对象	最终决定	本版执行方式
compare/cost_fusion.py	完全保留	不改名、不移动、不拆分，不借本任务清理其内部结构。
compare 数学工具	移动	features.py、distances.py、cosine.py、kl.py、sparse_ot.py 原样移入 compare/_shared/。
pij/legacy/*	不处理	保持现在的下划线内部模块与导入关系，不建立 legacy/_shared。
registry	不改结构	key、类映射和注册入口全部保持；理论上无需改 registry 文件。
package export	不扩大	只在确有旧路径 import 时修正，不新增公共暴露。

3. Network：把 GRN 私有逻辑并回所属方法

3.1 当前问题

mignet_ce/networks/grn_state.py 本身不是 registry 中可选择的 network method，但其内容不是通用 Network 工具，而是 light_cci_grn 构造 GRN 状态时使用的整套私有实现，包括：

- PreparedGRN 与 GRNStateResult；
- GRN 输入读取、基因与表达对齐；
- 双端表达门控；
- 确定性随机投影；
- projected GRN state 的构造与 metadata 写入。

它只被 light_cci_grn.py 直接使用；joint_cci_grn.py 对这套逻辑的使用来自继承链，而不是一个平行的通用调用者。因此其所有者仍然是 light_cci_grn.py。

3.2 目标修改

将 grn_state.py 的代码原样迁入 light_cci_grn.py。推荐的文件内部顺序为：

1. import；
2. GRN 相关 dataclass；

3. 输入准备和对齐函数；
4. 双端表达门控函数；
5. 投影和 state 构造函数；
6. LightCCIGRNNetworkBuilder 类。

迁移后删除：

```
mignet_ce/networks/grn_state.py
```

并移除 `light_cci_grn.py` 中对旧模块的 `import`。`joint_cci_grn.py` 的继承关系、类名、registry key 与行为不变。

本步骤只改变代码位置。随机投影的 seed、哈希规则、矩阵维度、双端门控公式、稀疏矩阵类型、metadata key 和异常处理必须逐行保持。

4. PIJ：把 block KL 交还 compare_N_kl

4.1 为什么 block KL 是方法私有逻辑

当前 `block_kl.py` 虽然由 `common.py` 调用，但触发条件实际绑定到：

```
self.pij_key == "kl"  
and self.feature_keys == ("N",)  
and feature_set.metadata["grn_block"]["enabled"]
```

这意味着它只改变注册方法 `compare_N_kl` 在 GRN block 开启时的行为。用户打开 `compare_N_kl.py` 时却只能看到方法名、特征键和距离类型，看不到该方法最关键的特殊成本构造，这正是需要修复的归属问题。

4.2 最小化改造方式

`common.py` 不移动、不改名。只在现有 `ComparePijMethodBase` 中增加一个通用 KL cost hook，让普通方法继续使用当前实现：

```
class ComparePijMethodBase:  
    def build_kl_cost(  
        self,  
        source,  
        target,  
        *,  
        feature_set,  
        side,
```

```

        pair,
        **kwargs,
    ):
        return pairwise_feature_kl(source, target, beta=...)

```

随后让 `CompareNKLpijMethod` 覆盖这一 hook，并把 `block_kl.py` 的实现放在同一方法文件中：

```

class CompareNKLpijMethod(ComparePijMethodBase):
    name = "compare_N_kl"
    feature_keys = ("N",)
    pij_key = "kl"

    def build_kl_cost(self, source, target, *, feature_set, **kwargs):
        block_meta = feature_set.metadata.get("grn_block", {})
        if not block_meta.get("enabled", False):
            return super().build_kl_cost(
                source, target, feature_set=feature_set, **kwargs
            )
        return self._build_block_kl_cost(
            source, target, feature_set=feature_set, **kwargs
        )

```

原 `build_block_kl_cost()` 可改为该类的私有静态方法，也可作为 `compare_N_kl.py` 内部以下划线开头的模块函数。关键不是形式，而是它必须和所属注册方法处于同一文件。

4.3 必须保持的数值语义

- GRN block 关闭时，结果与普通 N-only KL 路径完全一致；
- `weight_g=0` 时继续走当前 N-only exact fallback；
- 双 block 开启时，仍按当前规则分别做稳健归一化后加权；
- diagnostics 的 key、含义和数值保持；
- 不改变 KL beta、epsilon、裁剪方式或候选边。

完成后删除：

```
mignet_ce/pij/compare/block_kl.py
```

5. Compare 数学工具：只移动，不重写

5.1 本步骤的准确边界

用户最终同意原方案中的“步骤四：移动共享数学模块”，但不接受把整个 PIJ 体系都改造成新 shared 架构。因此本版采用最小方案：

仅把确实属于纯数学、纯特征构造或纯优化器的五个文件放入 `compare/_shared/`。 `common.py`、`cost_fusion.py`、所有注册方法文件和 `legacy` 文件都不进入该目录。

目标移动如下：

```
mignet_ce/pij/compare/features.py
-> mignet_ce/pij/compare/_shared/features.py

mignet_ce/pij/compare/distances.py
-> mignet_ce/pij/compare/_shared/distances.py

mignet_ce/pij/compare/cosine.py
-> mignet_ce/pij/compare/_shared/cosine.py

mignet_ce/pij/compare/kl.py
-> mignet_ce/pij/compare/_shared/kl.py

mignet_ce/pij/compare/sparse_ot.py
-> mignet_ce/pij/compare/_shared/sparse_ot.py
```

并新增一个尽量空的：

```
mignet_ce/pij/compare/_shared/__init__.py
```

5.2 为什么本版不再合并 `distances/cosine/kl`

原方案建议把三个距离文件合并为一个 `distances.py`。最新反馈强调“不需要做太多结构改造”，因此本版取消文件合并，只做路径移动。这样有三个好处：

- diff 更小，回归问题更容易定位；
- 函数名、模块边界和测试引用基本不变；
- 不会在一次所有权重构中混入距离 API 重设计。

5.3 允许修改的内容

只允许：

- 修改 import 路径；
- 处理移动后必要的相对/绝对导入；
- 为避免循环 import 做最小顺序调整；
- 删除移动前的旧文件。

不允许重命名函数、合并公式、改变 dtype、修改归一化、优化向量化实现或调整异常条件。

6. 明确保持原样的部分

6.1 common.py

`mignet_ce/pij/compare/common.py` 保持当前名称和路径。它继续承担普通 `compare` 方法的统一执行框架。除 block KL hook 所需的最小修改外，不做：

- 改名；
- 拆文件；
- artifact 导出函数迁移；
- 统一方法配置重构；
- 大规模清理分支。

6.2 cost_fusion.py

`mignet_ce/pij/compare/cost_fusion.py` 完全保留。十个 `costmix/euclidean` 方法继续继承当前类，不改文件名、不改 import 语义，只需要在其内部引用数学工具时更新为新的 `_shared` 路径。

6.3 legacy

`mignet_ce/pij/legacy/_ot_common.py` 与 `mignet_ce/pij/legacy/_developmental_ot_components.py` 已经通过下划线表达内部模块，并被多个 legacy 方法使用。本轮不移动、不改名、不建立 `legacy/_shared`，也不清理 legacy 导入。

6.4 registry 与 package export

`networks/registry.py` 和 `pij/registry.py` 的 key、映射类、导入入口都不应变化。由于本轮移动的 PIJ 数学文件本来就不是 registry 入口，正常情况下 registry 文件无需编辑。

`__init__.py` 只在存在对旧数学工具路径的显式导出时做必要修正；不借此把 `_shared` 中的内容重新公开到 package 顶层。

7. 修订后的目标目录

7.1 Network

```
mignet_ce/networks/
|-- __init__.py
|-- base.py
|-- registry.py
|-- light_cci.py
|-- light_cci_grn.py      # 内含原 grn_state.py 私有逻辑
|-- joint_cci_grn.py
`-- ... 其他已注册 network 方法 ...

# 删除 grn_state.py
```

7.2 PIJ compare

```
mignet_ce/pij/compare/
|-- __init__.py
|-- common.py            # 保持名称；仅增加通用 KL hook
|-- cost_fusion.py       # 保持名称与职责
|-- compare_N_kl.py      # 内含 block KL 私有实现
|-- compare_E_cos.py
|-- compare_L_sot.py
|-- ... 其他注册方法文件 ...
|-- compare_main_lap_sr_spatial_sot.py
`-- _shared/
    |-- __init__.py
    |-- features.py
    |-- distances.py
    |-- cosine.py
    |-- kl.py
    `-- sparse_ot.py

# 删除 block_kl.py
```

```
# 删除 compare 根目录下五个数学工具的旧副本
```

7.3 PIJ legacy

```
mignet_ce/pij/legacy/  
|-- _ot_common.py  
|-- _developmental_ot_components.py  
`-- ... 所有现有 legacy 方法文件 ...  
  
# 本轮完全不改变
```

8. 文件级 Codex 实施步骤

8.1 步骤 0：建立重构前快照

在改动前保存：

1. `NETWORK_BUILDERS.keys()` 排序结果；
2. `PIJ_METHOD_REGISTRY.keys()` 排序结果；
3. 每个 key 对应的类名与 name；
4. 关键默认配置；
5. 小型 fixture 上代表方法的 cost、P 矩阵和 metadata。

代表方法至少包括：`light_cci_grn`、`joint_cci_grn`、`compare_N_kl`、`compare_L_sot`、一个 `costmix` 方法和一个 legacy OT 方法。

8.2 步骤 1：合并 Network GRN 私有逻辑

1. 将 `grn_state.py` 的 `dataclass` 与函数原样复制到 `light_cci_grn.py`；
2. 删除旧 import；
3. 删除 `grn_state.py`；
4. 全仓库搜索旧路径；
5. 对 `light_cci_grn` 和 `joint_cci_grn` 做数值回归。

8.3 步骤 2：把 block KL 交还 compare_N_kl

1. 在 `common.py` 增加默认 `build_kl_cost()` hook;
2. 让普通 KL 路径通过该 hook 调用原有 KL 实现;
3. 在 `compare_N_kl.py` 覆盖 hook;
4. 将 block KL 函数原样迁入该方法文件;
5. 删除 `common.py` 中针对 `feature_keys==("N",)` 的隐藏特判;
6. 删除 `block_kl.py`;
7. 分别测试 block 开启、block 关闭与 `weight_g=0`。

8.4 步骤 3：移动五个数学工具文件

1. 创建 `compare/_shared/` 与空的 `__init__.py`;
2. 原样移动 `features.py`、`distances.py`、`cosine.py`、`kl.py`、`sparse_ot.py`;
3. 更新 `common.py`、`cost_fusion.py`、主方法和其他直接调用者的 `import`;
4. 不合并文件，不改函数签名;
5. 删除 `compare` 根目录的旧副本。

8.5 步骤 4：只清理本轮产生的旧路径

清理范围限定为已删除或已移动文件：

```
grep -R "networks.grn_state" -n mignet_ce scripts tests

grep -R "pij.compare.block_kl" -n mignet_ce scripts tests

grep -R "pij.compare.features" -n mignet_ce scripts tests
grep -R "pij.compare.distances" -n mignet_ce scripts tests
grep -R "pij.compare.cosine" -n mignet_ce scripts tests
grep -R "pij.compare.kl" -n mignet_ce scripts tests
grep -R "pij.compare.sparse_ot" -n mignet_ce scripts tests
```

不搜索并替换 `pij.compare.common`、`pij.compare.cost_fusion` 或 `legacy` 路径，因为它们本轮保持不变。

8.6 步骤 5：验证 registry，而不是重构 registry

本步骤只做检查，不做结构设计：

```
assert sorted(before_network_keys) == sorted(after_network_keys)
assert sorted(before_pij_keys) == sorted(after_pij_keys)
assert before_key_to_class_name == after_key_to_class_name
```

若 registry 文件因间接 import 无法加载，优先修复被移动模块的 import 路径，不能通过改 key 或删除方法绕过。

9. 测试计划

9.1 导入与编译测试

```
python -m compileall -q mignet_ce scripts
python -c "import mignet_ce"
python -c "from mignet_ce.networks.registry import NETWORK_BUILDERS"
python -c "from mignet_ce.pij.registry import PIJ_METHOD_REGISTRY"
```

还需遍历所有 registry key 完成实例化，以捕获移动数学模块后产生的漏改 import 和循环 import。

9.2 Network 数值回归

固定输入和 seed，比较重构前后：

- GRN 对齐后的基因顺序；
- 双端门控矩阵；
- deterministic projection matrix；
- grn_state_csr 的 shape、indices、indptr 与 data；
- light_cci_grn 和 joint_cci_grn metadata。

纯位置迁移原则上优先要求 bitwise equality。

9.3 compare_N_kl 回归

至少覆盖三种场景：

1. 没有 GRN block：应等于普通 N-only KL；
2. GRN block 开启且两块都有权重：总 cost、分块统计与 P 矩阵一致；
3. weight_g=0：exact fallback 与原结果一致。

9.4 数学工具移动回归

至少验证：

- 一个 cosine 方法；
- 一个 KL 方法；
- 一个 SOT 方法；
- 一个 costmix cosine 方法；
- 一个 costmix Euclidean 方法；
- `compare_main_lap_sr_spatial_sot`。

移动文件不应造成任何公式、dtype、稀疏结构或 diagnostics 差异。

9.5 legacy 保护性测试

虽然 legacy 不做结构修改，仍应选一个 legacy OT smoke test，确认 compare 模块移动没有通过公共 import 意外影响 legacy。该测试是保护性检查，不代表整理 legacy。

10. Codex 不得做的事情

1. 不得把 `common.py` 或 `cost_fusion.py` 改名、移动或拆分；
2. 不得创建 `legacy/_shared`，不得移动 legacy 内部模块；
3. 不得合并 `distances.py`、`cosine.py` 和 `kl.py`；
4. 不得把数学工具复制进每个方法文件；
5. 不得改变任何 network/PIJ registry key 或删除薄方法入口；
6. 不得借 block KL 归属调整修改 KL 公式、权重或归一化；
7. 不得借 GRN 文件合并修改门控、投影、维度或 metadata；
8. 不得扩大 `__init__.py` 的公共导出面；
9. 不得顺手清理 legacy、velocity OT、历史方法或实验矩阵；
10. 不得在本任务中做性能优化和大规模命名统一。

11. 完成标准

本次重构同时满足以下条件才算完成：

1. `grn_state.py` 已删除，其全部私有逻辑能在 `light_cci_grn.py` 直接阅读；
2. `block_kl.py` 已删除，`compare_N_kl.py` 明确拥有 block KL 行为；
3. `common.py` 与 `cost_fusion.py` 的文件名和路径保持；
4. 五个数学工具已原样移动到 `compare/_shared/`，没有文件合并；
5. `legacy` 目录结构和导入路径没有变化；
6. `registry` key、类映射、默认配置和可实例化性完全一致；
7. 关键方法的 `cost`、`P` 矩阵、`metadata` 与导出语义回归通过；
8. 全仓库没有本轮被删除/移动路径的残留 `import`；
9. `compileall`、`registry import` 与小型 `smoke test` 全部通过。

12. 最终建议

这一版不再追求“目录看起来绝对统一”，而是只处理真正妨碍方法可追踪性的对象：

- 方法私有业务逻辑回到方法文件：GRN state 回到 `light_cci_grn`, block KL 回到 `compare_N_kl`；
- 已有执行框架保持：`common.py` 和 `cost_fusion.py` 不动；
- 纯工具只做轻量归档：五个数学文件移到最小化的 `_shared`，但不重写、不合并；
- 旧体系不扩大改动：`legacy` 和 `registry` 只验证，不重构。

一句话概括：能明确归属于单一方法的逻辑就并回方法；已经稳定工作的公共执行框架保持原样；真正无业务归属的数学工具只移动目录，不改变实现。